Modeling Horse Race Outcomes — HKJC

Project Methodology & Production Notes

April 20, 2026

Contents

1 Introduction **1**

2 Horse Racing Context and HKJC Data **1**

2.1 Race Structure 1

2.2 Variables used 2

2.3 Data hygiene 2

3 Data Representation and Notation **2**

4 Modeling Approaches **3**

4.1 Binary Logistic-Loss Boosted Trees 3

4.2 Exploded Logit (Plackett–Luce) 3

4.3 Softmax (Winner-Only) 3

4.4 Offset Model (Production) 3

5 Production Feature Set (25 Features) **4**

5.1 Market (2) 4

5.2 Race card (4) 5

5.3 Class & Weight (3) 5

5.4 Form features (4) 5

5.5 Trainer features (3) 5

5.6 Draw & field interactions (2) 5

5.7 Barrier trial features (2) 6

5.8 Sectional features (4) 6

5.9 Body weight residual (1) 6

6 Architecture: 5-Seed Bagging **7**
6.1 Method . . . 7
6.2 Effect . . . 7

7 Kelly Bet Sizing **8**
7.1 Multinomial Kelly . . . 8
7.2 Rebate . . . 8
7.3 Kelly sweep . . . 8

8 Feature-Engineering Methodology **9**
8.1 Testing protocol . . . 9
8.2 Design principles . . . 9
8.3 What was tried and rejected . . . 10
8.4 What worked . . . 11

9 Implementation Details **11**
9.1 Preparing group pointers . . . 11
9.2 Handling ties and missing positions . . . 11
9.3 Time-respecting cross-validation . . . 11

10 Results: Year-by-Year (2015–2026) **11**
10.1 Market vs model log-loss . . . 11
10.2 Year-by-year ROI, Kelly 0.02 . . . 11
10.3 Observations . . . 12

11 Practical Concerns **12**
11.1 Edge decay over time . . . 12
11.2 Rebate dependency . . . 13
11.3 Bet-time feature stability . . . 14
11.4 Strategic jockey behaviour . . . 14

12 Reproducing the Pipeline **14**

1 Introduction

This document covers the full methodology of the HKJC horse-racing probability model, from first principles through the current production configuration. It unifies three strands of work:

1. **Modeling frameworks** — binary logistic, exploded logit, softmax, and the market-anchored *Offset* model that is the current production choice.
2. **Feature engineering** — the 25 features used in production, why each is constructed the way it is, and what alternatives were tested and rejected.
3. **Betting & results** — Kelly bet sizing, bagging-based variance reduction, and cumulative performance over 2015–2026 (including out-of-sample 2025 and partial 2026).

The production recipe, in one line: **5-seed bagged Offset model with 25 features (including the `body_wt_mkt_residual` feature), trained on all history prior to the test year, bet with Kelly fraction 0.02 on a $10M bankroll**.

Cumulative ROI over 2015–2026 at Kelly 0.02: **+332.5% without rebate, +880.1% with 10% rebate on race-losses $\geq$ $10k**, with 9/12 and 11/12 positive years respectively.

2 Horse Racing Context and HKJC Data

2.1 Race Structure

Each race is classified by:

- **Distance** — 1000m, 1200m, 1400m, 1600m, 1650m, 1800m, 2000m, 2200m. Distances vary by racecourse and day.
- **Surface** — *Turf* or *AWT* (all-weather track).
- **Class tier** — Class 1 (toughest) through Class 5, or Group races. Class is handicap-rating-banded.
- **Racecourse** — Sha Tin (ST, larger) or Happy Valley (HV, tight 1.2km circumference).

2.2 Variables used

For each horse in each race, HKJC publishes (pre- and post-race):

- Identity: `horse_id`, trainer, jockey.
- Structural: `Draw` (stall position), `Act_Wt` (weight carried), `Declar. Horse Wt.` (declared body weight), `Gear`.
- Market: `Win Odds` (final). Implied probability $p_{\text{implied}} = 1/\text{odds}$.

- Outcome: `Pla.` (finish position), `Finish Time`, `Running Position` (positions at 3–6 intra-race sections).

Additional data we scrape separately:

- **Barrier trials** (pre-race gallops, 53k+ records).
- **Sectional times** (per-section times and positions, 648k rows).
- **Dividend information** for refunds and payout verification.

2.3 Data hygiene

- Refunded runners (`WV`, `WV-A`, `WX`, `WX-A`, `WXNR`) are dropped entirely.
- Void finishers (`PU`, `DNF`, `DISQ`, `UR`, `FE`, `TNP`) are coded 99 and excluded from training.
- Races with gappy or duplicated positions are removed pre-training (see `remove_tied_entries`).

3 Data Representation and Notation

Each observation is indexed by (race_id, horse_id). Let R = number of races, m_r = field size of race r, and $N = \sum_r m_r$ = total observations.

Feature matrix $X \in \mathbb{R}^{N \times p}$; for observation i, feature vector $x_i \in \mathbb{R}^p$ ($p = 25$ in production).

Outcomes:

- Binary win indicator $y_i^{\text{win}} \in \{0, 1\}$; exactly one horse per race has $y^{\text{win}} = 1$.
- Ordinal finish position $y_i^{\text{place}} \in \{1, \ldots, m_r\}$.

4 Modeling Approaches

We developed three gradient-boosting objectives and evaluated a fourth architecture (Offset) that became production.

4.1 Binary Logistic-Loss Boosted Trees

Each horse treated as a separate binary example. Given $F(x_i) \in \mathbb{R}$, $p_i = \sigma(F(x_i))$ where $\sigma(z) = 1/(1 + e^{-z})$.

$$\mathcal{L}_{\text{logistic}}(F) \;=\; \sum_{i=1}^{N} \Big[-y_i^{\text{win}} \log p_i \;-\; (1 - y_i^{\text{win}}) \log(1 - p_i) \Big].$$

Gradient / Hessian: $\partial \ell / \partial F = p_i - y_i$, $\partial^2 \ell / \partial F^2 = p_i(1 - p_i)$.

Ensemble: $F_T(x) = \sum_{m=1}^{T} \nu\, h_m(x)$, each h_m a regression tree of depth $\leq D$ trained on pseudo-residuals.

Post-hoc calibration: `CalibratedClassifierCV` (isotonic) on an inner cross-validated fold, then per-race softmax normalization so probabilities sum to 1 within a race.

4.2 Exploded Logit (Plackett–Luce)

Models the full ranking of horses in a race. For race r with latent scores $f_i = F(x_i)$ and true ranking positions y_i^{place}, the likelihood of the observed ordering is:

$$\mathcal{L}_r = \prod_{k=1}^{K} \frac{\exp(f_{\pi_k})}{\sum_{j \in \text{remaining at stage } k} \exp(f_j)}$$

where π_k is the horse in position k. With truncation at top-n (winner-only when $n = 1$, top-3 when $n = 3$, full ordering when $n = -1$).

Negative log-likelihood, gradient, and Hessian are coded into a custom XGBoost/LightGBM objective (`exploded_logit_obj`).

4.3 Softmax (Winner-Only)

Equivalent to exploded logit with $n = 1$: only the winner's relative probability is modeled. Trained with a custom softmax objective (`softmax_obj`).

Post-hoc calibration for softmax uses binned temperature scaling (per field-size bin, optionally per surface).

4.4 Offset Model (Production)

This is the current production model. Instead of learning the log-odds from scratch, the Offset model anchors predictions to the market and learns only the *residual*.

Let π_i denote the normalized market implied probability for observation i. Define the market log-odds

$$\eta_i^{\text{mkt}} = \text{logit}(\pi_i) = \log \frac{\pi_i}{1 - \pi_i}.$$

We train a binary-logistic XGBoost with η_i^{mkt} as a fixed `base_margin`:

$$p_i = \sigma\big(\eta_i^{\text{mkt}} + F(x_i)\big),$$

where $F(\cdot)$ is the XGBoost ensemble. When $F(x_i) = 0$, the model reproduces the market. XGBoost's rounds adjust only $F(x_i)$, the *correction* to market.

Why this helps.

1. The market is an excellent calibrated probability estimator. Asking the model to re-derive it from scratch wastes tree capacity.

2. **Importance is evenly distributed across edge features** (gain $\approx$ 4–6) rather than concentrated on the market anchor. This reveals which features are actually contributing residual signal.
3. Post-hoc isotonic calibration *hurts* Offset — the market anchor is already calibrated; reshaping the output distorts it. See Section 8.3.

Offset hyperparameters (deep_slow, tuned).

- `learning_rate=0.01`, `max_depth=5`
- `min_child_weight=20`
- `subsample=0.7`, `colsample_bytree=0.7`
- `gamma=0.2`, `reg_alpha=2.0`, `reg_lambda=10.0`
- `rounds=400`

Depth 5 is deeper than typical softmax configs because the model only learns corrections — deeper trees capture feature interactions without overfitting the market signal.

5 Production Feature Set (25 Features)

All features are *time-respecting*: any expanding / rolling statistic uses only past races (shift-by-1 or expanding-minus-current).

5.1 Market (2)

- `Implied_Prob` = 1/Win Odds, renormalized per race.
- `log_implied_prob` = log(Implied_Prob).

5.2 Race card (4)

- `Racecourse` — ST=1, HV=0.
- `Track_Turf` — Turf=1, AWT=0.
- `Draw` — stall number 1–14.
- `Act_Wt` — weight carried, lbs.

5.3 Class & Weight (3)

- `class_change` = prev_class − current_class. The most important single feature by XGBoost gain (rank 1–2 nearly every year). Captures difficulty-level change.
- `wt_z` — z-score of `Act_Wt` within the current field.
- `setup_weight_z` — z-score of `Act_Wt` vs horse's own expanding history of carried weights.

5.4 Form features (4)

- `recent_form` = avg(1/place) over last 3 races. Concave weighting: 1st → 1.0, 2nd → 0.5, 3rd → 0.33, tapering for back-of-pack. Empirically outperforms linear normalization.
- `form_vs_career` = recent_form − career_form. Trajectory signal.
- `career_beat_odds` — expanding mean of $(y_i^{\text{win}} - \pi_i)$ per horse. A per-horse market-residual.
- `prev_win` — binary: won last race.

5.5 Trainer features (3)

- `trainer_track_spec` — trainer's win rate at this specific racecourse minus trainer's overall win rate. Only applied with min-runs thresholds (50 overall, 20 per track).
- `tt_gap` — trainer × track actual WR minus market-implied WR.
- `tr_gap` — trainer overall actual WR minus market-implied WR.

5.6 Draw & field interactions (2)

- `draw_outside_ST` — binary: `Draw` ≥ 10 **and** `Racecourse`=ST. Outside-draw penalty at Sha Tin (except ST 1000m straight, where it reverses — the tree learns the reversal from other features).
- `fav_field_size` = `is_fav` × `field_size`. Captures the small-field favourite overpricing effect.

5.7 Barrier trial features (2)

Market-orthogonal signals from pre-race gallops.

- `bt_avg_early` — expanding average of early_pos/field_size in past barrier trials. Running-style signature.
- `bt_last_behind` — time behind winner in the most recent barrier trial.

5.8 Sectional features (4)

Computed from per-horse-per-section times.

- `trail_sect_closing` — avg closing speed vs race average (last-section speed differential), trailed across past races.
- `trail_sect_peak_at` — average position (0=early, 1=late) of horse's peak sectional speed. Running-style signature.
- `trail_sect_avg` — overall sectional-speed deviation vs race average.
- `trail_sect_best_gain` — biggest intra-race position gain in one section, averaged across races. Explosive-move capability.

5.9 Body weight residual (1)

The newest and most impactful feature added this session.

Construction. For each row, define

$$\text{bucket} = (B_{\Delta w}, B_{\pi}),$$

where $B_{\Delta w}$ bins body_wt_change into $\{< -10, [-10, -5], [-5, +5], [+5, +10], > +10\}$ lbs, and B_{π} bins `Implied_Prob` into $\{< 0.04, [0.04, 0.08], [0.08, 0.15], [0.15, 0.25], > 0.25\}$. That yields up to $5 \times 5 = 25$ buckets.

Time-respecting expanding statistics per bucket:

$$\text{past_wins}_i = \sum_{j<i,\ \text{bucket}(j)=\text{bucket}(i)} y_j^{\text{win}}, \qquad \text{past_implied}_i = \sum_{j<i,\ \text{bucket}(j)=\text{bucket}(i)} \pi_j.$$

The feature is a shrunk residual:

$$\texttt{body_wt_mkt_residual}_i = \frac{\text{past_wins}_i - \text{past_implied}_i}{\text{past_implied}_i + k}, \qquad k = 50.$$

This is the **market-failure residual**: it measures, bucket by bucket, the historical deviation between actual wins and market-implied wins. Positive ⇒ market systematically underprices this (body_wt_change × implied_prob) combination; negative ⇒ systematically overprices.

Empirical signal. The strongest effects are:

- Favourites ($\pi > 0.15$) with moderate weight drops or gains: +2 to +3% edge.
- Horses after long rest (>60 days): weight *gain* is positive (horse stronger).
- Volatile-weight horses with big drops: +0.7% edge.
- Previous winners with weight drops: +1.3% edge.

Why this specifically succeeded. Earlier sessions tried many "target market-failure residual" features (e.g., draw-bias residual, class-change residual); most hurt. Two distinguishing factors for body weight:

1. `Declar. Horse Wt.` is nearly orthogonal to market odds (correlation ≈ 0.01). The market has minimal pricing of body-weight dynamics.
2. Effects are strongly non-monotonic and interaction-dependent (different direction by implied-prob bucket). A single raw feature cannot capture this; a bucketed residual can.

Across 8 seeds: mean paired $\Delta = +34$pp cumulative ROI over the baseline, 7/8 seeds positive, minimum-case ROI compressed from −33% to −3%.

6 Architecture: 5-Seed Bagging

6.1 Method

For each test year, train 5 independent Offset models with different XGBoost random seeds, $s \in \{7, 17, 42, 123, 256\}$. Each model sees the same data but makes different random choices in column subsampling, row subsampling, and split tie-breaking. Predictions are averaged:

$$\hat{p}_i^{\text{bagged}} = \frac{1}{5} \sum_{s=1}^{5} \hat{p}_i^{(s)},$$

then renormalized per race so probabilities sum to 1.

6.2 Effect

- **Log-loss unchanged** (mean ≈ 0.2452 across seeds and bagged). Bagging does not improve prediction accuracy.
- **Prediction variance reduced** — average per-horse probability becomes more stable.
- **Kelly amplification of noise** drops — bagged bet sizes are more consistent race-to-race.
- Cumulative ROI in a 3-meta-seed test: mean +71% vs single-seed mean +49% (over the 2015–2024 window). Min cum ROI across meta-seeds went from −3% (single-seed) to +41% (bagged). **Downside effectively eliminated.**
- Historically losing years 2018 and 2019 flipped from negative to positive under bagging.

5 seeds is a good point on the diminishing-returns curve: further seeds add marginal stability at 5× compute per additional set.

7 Kelly Bet Sizing

7.1 Multinomial Kelly

For a race with field size m, probabilities $p_1, \ldots, p_m$ (model), payoffs $b_1, \ldots, b_m$ (decimal odds), Kelly maximises expected log-wealth:

$$\max_{w_1,\ldots,w_m \geq 0,\ \sum w_k \leq 1} \sum_k p_k \log\left(1 - \sum_j w_j + w_k b_k\right).$$

In practice we scale the optimal fractions by α (*fractional Kelly*) to reduce variance: $w_k \to \alpha w_k$. Production uses $\alpha = 0.02$.

Constraints in the `multinomial_kelly` implementation:

- `max_per_horse` = 15% of bankroll.
- `max_total` = 30% of bankroll per race.
- Minimum bet \$10; sub-minimum stakes are rounded up or dropped.

7.2 Rebate

HKJC offers a rebate of 10% on race-level losses at or above \$10k for high-volume players. The simulation reports *both* trajectories: "without rebate" (raw Kelly P&L) and "with rebate" (adds $0.1 \times \max(0, \text{race_loss} - 10{,}000)$ back to the bankroll at the end of each race where the condition triggers).

7.3 Kelly sweep

At Kelly fractions $\{0.01, 0.02, 0.03, 0.05\}$:

Metric	K=0.01	K=0.02	K=0.03	K=0.05
Cum ROI (no rebate)	+139.5%	**+332.5%**	+522.7%	+644.8%
Cum ROI (with rebate)	+260.3%	+880.1%	+2027.3%	+5701.8%
Positive years (no rebate)	9/12	**9/12**	7/12	7/12
Positive years (with rebate)	11/12	**11/12**	10/12	9/12

Kelly 0.02 is the recommended production setting: 9/12 positive years without rebate, 11/12 with rebate, and the best Sharpe-like balance. Kelly 0.05 has higher mean but larger drawdowns in bad years.

8 Feature-Engineering Methodology

This section captures the discipline that produced the current feature set and the design principles learned along the way.

8.1 Testing protocol

1. **Empirical bucket analysis first.** Before proposing a new feature encoding, measure the raw effect across buckets on historical data. Look for:
   - Effect size ($|\text{edge}| \geq 0.5\%$ is a minimum bar).
   - Year-by-year consistency (11+/14 years sameside is strong).
   - Non-monotonicity or bucket-specific direction.
2. **Paired-delta test across ≥ 3 seeds.** Single-seed ROI differences are noise-dominated at the scale of single-feature changes. Require:
   - Mean paired $\Delta > 0$ with all seeds same-sign, OR
   - If mean is positive but some seeds negative, suspect regression-to-the-mean on lucky seeds.
3. **Log-loss as primary predictive metric.** LL is stable across seeds (std $\sim 5 \times 10^{-5}$); ROI is high-variance (std $\sim$ 15–35%). If LL is unchanged, the feature doesn't improve predictions — ROI swing is Kelly amplification of noise.

4. **Multi-year ROI as production-relevant metric.** LL is what we're modeling, ROI is what we care about. Both must move to declare a win.

8.2 Design principles

1. **Target market-failure residuals, not market-known structure.** The market already prices known effects (draw bias, class drops, favourite-longshot). Encoding these as features duplicates information already in the offset's `base_margin` and adds only noise.

   Example failure: `draw_bias_loc` was a time-respecting bucketed lift feature for draw. Empirically accurate; in-model it cost −21pp cumulative ROI because the market already knows.

2. **Prune low-importance leaf features; preserve backbone categoricals.** Low gain-importance can mean two things:

   - Genuine dead weight (leaf feature with no interaction role) — safe to prune.
   - Backbone categorical (`Racecourse`, `Track_Turf`) that enables *other* features' interactions — dropping crashes the model.

   Rule: if any other feature's computation or interaction depends on this one, do not prune.

3. **Don't re-encode top-importance features.** If a feature consistently ranks 1–3 in gain, XGBoost has already found optimal splits. Adding a derived version (e.g., `class_change_residual` built from `class_change`) wastes tree capacity on correlated information.

4. **Shape matters for form encodings.** `recent_form` using 1/place (concave, emphasising wins) outperforms linear normalization by (field_size − place + 1)/field_size. Horse quality is better signalled by top finishes than by average position.

5. **Re-engineering payoff is concentrated at mid-to-low importance.** Top features are already optimal; dead features are dead. The sweet spot for investment is features at gain-rank 5–15 that have known market-failure structure.

8.3 What was tried and rejected

Full list of re-engineering attempts that failed, saved here so future sessions don't repeat the work:

1. `draw_bias_loc`: bucketed draw-bias residual. −21pp. *Reason:* market already prices draw bias.

2. `class_change_residual`: bucketed residual on (current_class × class_change). −38pp across seeds. *Reason:* `class_change` is already rank 1 in gain; re-encoding adds overfitting surface without info.

3. `prev_mkt_residual` swap for `prev_win`: neutral on LL, high-variance ROI. Mean +9pp but one seed crashed. *Kept `prev_win` binary.*

4. Form normalization (field_size − place + 1)/field_size replacing 1/place: −32pp. *Reason:* concave weighting of top finishes is the correct shape.

5. LOO on odds-dependent features (`fav_field_size`, `career_beat_odds`, `tt_gap`, `tr_gap`): every removal hurt −20 to −40pp.
6. Sectional consolidation (4 → 2 or 1): −22 to −47pp. *Reason:* all 4 features carry complementary information.
7. Pruning `Racecourse` or `trail_sect_closing`: −30 to −66pp. *Reason:* backbone for downstream interactions.
8. Isotonic post-hoc calibration on Offset: −141pp. *Reason:* market anchor via `base_margin` is already calibrated by construction; isotonic distorts the output.
9. Softmax + isotonic calibration (with `log_implied_prob`): −91pp. *Reason:* Softmax must learn the market anchor from scratch; fundamentally weaker than hard-anchoring via base_margin.
10. Bagged Offset + Softmax ensemble (50/50): −62pp. *Reason:* blending a strong model with a weaker one yields a mediocre model.
11. Kelly $\alpha = 0.10$: catastrophic loss (−92% cumulative). Volatility drag compounds across bad years.
12. Adding raw body weight features (`body_wt_z` + `body_wt_change`): −54pp. *Only the bucketed residual form works.*
13. `body_wt_after_win` targeted binary: −48pp. Too sparse.

8.4 What worked

1. **Prune2** — dropped `top3_count_5` and `field_form_cv` (lowest-gain leaf features). +69pp paired Δ, 3/3 seeds positive.
2. `body_wt_mkt_residual` — bucketed body-weight market-failure residual (Section 5.9). +34pp paired Δ across 8 seeds.
3. **5-seed bagging** — averaging Offset predictions across independent seeds. +22pp mean, downside compressed.
4. **Higher Kelly safe with bagging** — sweet spot shifted from 0.01–0.02 (single-seed) to 0.02–0.03 (bagged).

9 Implementation Details

9.1 Preparing group pointers

XGBoost requires group sizes (one integer per race, giving the number of horses). Our helper `prepare_group_sizes(X)` produces this from a `MultiIndex`-ed DataFrame.

9.2 Handling ties and missing positions

For exploded-logit training, ties are broken by averaging the negative-gradient penalty across tied horses. Missing positions (void finishers) are excluded before any shift/expanding computation.

9.3 Time-respecting cross-validation

Production uses expanding-window temporal split: train years = all years strictly before the test year. No fold contains future data. Separate calibration year (the last train year) is carved out only when `--calibrate` is explicitly set; not used for Offset.

10 Results: Year-by-Year (2015–2026)

10.1 Market vs model log-loss

Model beats market log-loss 10 of 10 years in the original test (2015–2024), mean $\Delta = -0.00089$ (excluding 2024 outlier: -0.00047). The absolute improvement is small ($\sim 0.2\%$ relative) but consistent.

10.2 Year-by-year ROI, Kelly 0.02

Bagged Offset, per-year reset to $10M, across 12 test years:

Year	ROI no-rebate	ROI with-rebate
2015	+82.35%	+106.97%
2016	+68.03%	+85.44%
2017	+2.26%	+11.46%
2018	+0.21%	+8.53%
2019	+7.81%	+15.65%
2020	−12.72%	−5.75%
2021	+24.78%	+33.27%
2022	+12.72%	+18.54%
2023	+7.58%	+12.75%
2024	−4.98%	+1.49%
2025	−2.83%	+0.94%
2026*	+4.77%	+6.11%
Cumulative	+332.5%	+880.1%
Positive years	9/12	11/12

*2026 is partial (approximately 2,500 races vs 9,000 full year).

10.3 Observations

- **2015–2016** produced the bulk of the cumulative alpha. Market LL in those years was low (market was sharp) — the model still beat it handily.
- **2020** is universally the worst year. Plausibly COVID-related disruptions (empty stadiums, jockey pool changes, betting-volume shift to online).

- **Recent years (2023–2025)** show smaller edges, consistent with the "market-efficiency edge decay" hypothesis: as more sophisticated bettors arbitrage the same signals, our market-failure residuals shrink.
- **Out-of-sample 2025 and 2026**: slight no-rebate loss in 2025, positive in 2026 partial. Both positive with rebate. The model appears to generalize to post-training periods.

11 Practical Concerns

11.1 Edge decay over time

The market-failure features (`body_wt_mkt_residual`, `tt_gap`, `tr_gap`, `career_beat_odds`) are *structural* residuals that automatically shrink as the market prices their content. The feature still computes correctly year-by-year (because it uses expanding history), but the magnitude of past_wins − past_implied per bucket decreases, so Kelly bets smaller, so ROI shrinks.

This is the structural reason 2015–2016 were outsized: the market had not yet priced certain patterns.

Truncated-training experiment (tested, rejected)

To test whether the drift was severe enough to justify reducing the training window, we backtested three windows across 2015–2026 at three seeds:

Window	Mean cum no-rebate	With rebate	Seeds no-rebate
Full (2010–$t-1$)	**+103.34%**	**+382.42%**	+26.6%, −18.9%, +302.4%
Last 5 years	−71.37%	+18.26%	−69.2%, −72.7%, −72.2%
Last 3 years	−85.06%	−1.42%	−67.2%, −91.1%, −96.9%

Truncation catastrophically hurt overall. Full history wins by a huge margin on cumulative ROI.

However, the per-year breakdown shows *recency-specific* gains hidden inside the global failure:

- 2024: last_3 was +7.92% vs full −4.46% (flipped positive).
- 2025: last_3 was +21.00% vs full −7.03% (large flip).
- 2026: last_3 was +3.12% vs full −0.30% (flipped positive).

So the edge-decay story has **partial support**: very recent years DO benefit from a shorter training window. But the cumulative cost in 2015–2021 (where full history is massively better) dwarfs the recency gain.

Implications for future work

- Simple truncation is too aggressive. Too much signal lives in older data.

- **Recency-weighted training** — weight recent races more ($w = \exp(-\lambda \cdot \text{years_ago})$) without discarding older data — is the natural next attempt. Untested.
- **Full-plus-truncated ensemble** — average predictions from a full-history model and a last-3-year model. Untested.
- Regular re-screening of feature importance by year to catch decaying features early.

11.2 Rebate dependency

Before body_wt + bagging: with-rebate was +78% vs no-rebate +14% — almost all the alpha came from rebate. After body_wt + bagging: no-rebate cumulative is +332.5%, with-rebate +880.1% — alpha is genuine, rebate adds significant further compounding but isn't load-bearing.

Critical for production: the strategy works without rebate, so is deployable for any player, though the tail compounds much higher with rebate.

11.3 Bet-time feature stability

Several features (`fav_field_size`, `career_beat_odds`, `tt_gap`, `tr_gap`, `body_wt_mkt_residual`) depend on odds observed pre-race. At actual bet time, odds can shift due to late market flow. LOO tests showed every odds-dependent feature earns its keep, so we keep them despite the fragility risk; production deployment should re-run inference shortly before bet placement.

11.4 Strategic jockey behaviour

Several features assume independent race-to-race horse behavior (`career_beat_odds`, form aggregates). If connections strategically rest a horse's form for later, these backward-looking features miss the signal. We have no observable signal for this intentionality beyond `Gear` and `class_change`.

12 Reproducing the Pipeline

Step 1: Build the feature parquet.

```
python data-processing/prep-data.py
```

Loads raw CSVs and JSONs, merges horse history, computes all 25 production features (including the body-weight market-failure residual), and writes `data/processed/race_features.parquet`.

Step 2: Run the production backtest + viz.

```
python model/run_production.py
```

Trains 5-seed bagged Offset for each test year 2015–2026, simulates Kelly $0.01, 0.02, 0.03, 0.05$ bankroll trajectories, and writes `model/results/bankroll_viz.html` — an interactive visualization with per-Kelly and per-year tabs.

Streamlit UI.

```
cd model && streamlit run app.py
```

Defaults on the Training page are Offset + 5-seed bagging + the tuned deep_slow hyperparameters. The Bankroll page defaults to Kelly 0.02 and $10M starting bankroll.

Appendix: Notation reference

π_i	market implied probability for observation i
$\sigma(\cdot)$	sigmoid function
$F(x)$	XGBoost ensemble output (log-odds correction in Offset)
η_i^{mkt}	$= \text{logit}(\pi_i)$, market log-odds
y_i^{win}	binary win indicator
y_i^{place}	finish position
m_r	field size of race r
$N = \sum_r m_r$	total observations
k	shrinkage prior for market-failure residuals ($= 50$)
α	Kelly fraction